24-HOUR AI HACKATHON · CAMPUS DRIVE

Build Something That Couldn't Have Existed Two Years Ago

A unique idea. A modern stack. Twenty-four hours to prove you can build what nobody has built before.

FORMAT	DURATION	TEAM SIZE	OUTPUT
Open innovation	**24 hours, continuous**	**2 – 4 members**	**Live demo + code + pitch**

This document is the complete brief. Read all of it before Hour 0.

INSIDE

01 · THE THEME

One Sentence, One Filter

> **THE RULE**
> Your project must be something that could not realistically have been built two years ago. If a team from 2023 could have shipped it with the tools they had, it does not belong here.

This single constraint does most of the work for us. It quietly rules out yet another to-do list, another CRUD dashboard, another login page with a gradient. It pushes every team toward capabilities that only became practical recently: agents that take actions, models that see and hear, systems that run on-device, retrieval over private knowledge, and interfaces where the user simply describes what they want.

Judges are instructed to ask exactly one question when they arrive at your table: **“Could this have existed in 2023?”** If the honest answer is yes, the project scores low on originality no matter how polished the interface looks.

The second filter: remove the AI

Take the intelligence out of your project. Does it still work? If yes, the AI is decoration and you have built a normal app with a chatbot bolted to the side. The strongest submissions **collapse completely** without the model, because the model is doing something no amount of hand-written logic could do.

> **SAY THIS OUT LOUD AT KICK-OFF**
> Wrapping an API is not a project. Everyone in the room has the same API key. What separates you is everything you build *around* the model — the retrieval, the guardrails, the evaluation, the fallbacks, the memory, the interface.

02 · TRACKS

Pick a Lane So Ideas Don't Collapse Into Chatbots

Every team chooses one track at registration. Tracks exist to spread the room out — without them, roughly seventy percent of submissions become the same conversational assistant.

Track	What belongs here	What we're looking for
Agents & Automation	Systems that decide and act, not just answer. Multi-step workflows, tool use, long-running background agents.	Recovery when a tool fails. Knowing when to stop.
Multimodal	Vision, voice, video, sensors, documents. Anything where the input isn't clean typed text.	Two modalities genuinely working together.
AI for Bharat	Regional languages, code-mixed speech, low bandwidth, offline-first, low-end devices.	Works on a weak network and a cheap phone.
Developer Tooling	Things that make building software faster — review, testing, migration, debugging, docs.	Something you'd actually keep using on Monday.
Wildcard	Anything that fits nowhere above. You must justify in one line why it doesn't fit.	Genuine strangeness. Surprise us.

03 · THE STACK

Trending Tools You're Encouraged to Use

A menu, not a mandate. Use anything you like — but a stack from five years ago will draw questions. Mixing an unusual tool into a modern one is the easiest way to stand out.

Layer	Trending choices	Why it scores well
Models	Claude, GPT-class APIs, Gemini, Llama, Mistral, Qwen, Phi — plus small local models via Ollama or llama.cpp	Using a small local model where a big API would be lazy shows real judgement.
Agent layer	LangGraph, CrewAI, tool-calling loops written from scratch, MCP servers	Hand-rolled loops with retries often beat framework magic. Show the control flow.
Retrieval	pgvector, Qdrant, Chroma, LanceDB, hybrid BM25 + dense, re-rankers	Chunking strategy and re-ranking are where RAG projects are actually won.
Frontend	Next.js, SvelteKit, Astro, React Native / Expo, Tauri, streaming UI, WebRTC	Streaming responses and optimistic UI make a demo feel finished.
Backend	FastAPI, Hono, Bun, Go, Rust, edge functions, WebSockets, job queues	Anything long-running needs a queue. Judges will ask about it.
Data & infra	Supabase, Neon, Turso, Redis, Cloudflare Workers, Docker, Fly.io	A deployed URL beats a localhost demo every single time.
On-device	WebGPU, ONNX Runtime, transformers.js, WASM, Core ML, TFLite	Running inference in the browser is still rare enough to be memorable.
Observability	Token and cost tracking, latency traces, prompt versioning, eval harnesses	Almost nobody does this. The teams that do are the teams we shortlist.

READ THIS TWICE

The unfair advantage: build a tiny evaluation harness — twenty test cases, a score, re-run after every change. Ninety minutes of work. Roughly one team in thirty does it, and that team almost always wins the technical depth score.

04 · CONSTRAINTS

Rules That Force Originality

Every team must satisfy at least **two** of the following. Declare which two in your submission form.

- **Two models, not one.** Two different models or two modalities must genuinely co-operate — not one model called twice.
- **Degrade gracefully.** Some part of the flow must survive a dead network, a rate limit, or a model that returns nonsense. You will be asked to demonstrate this live.
- **Handle being wrong.** Show us what happens when the model is confidently incorrect. A visible confidence signal, a human-review path, or a refusal all count.
- **Not already the top Google result.** At Hour 2, search for your own idea described in one sentence. Write down the three closest existing products and one line on how you differ.
- **Cost ceiling.** Your project must run for a full day of realistic usage under a stated budget. Tell us the number and how you got there.

05 · TIMELINE

The 24 Hours, Hour by Hour

Checkpoints are compulsory. Miss two and you are not eligible for the final shortlist, regardless of what you demo.

Hour	What happens	Deliverable
00:00	Kick-off. Theme, tracks and rubric explained. Teams register their track.	Team registered
02:00	**Prior-art check.** Search for your own idea. Find the three closest existing products.	Three links + one line on how you differ
03:00	**Idea lock.** One-paragraph pitch submitted. Mentors approve or send back. After this hour, no pivots.	Approved pitch
06:00	**Checkpoint 1 — architecture.** Explain your design to a mentor in five minutes without opening your editor.	Architecture sketch
12:00	**The curveball.** A new requirement is announced to every team simultaneously. It is not optional.	—
18:00	**Checkpoint 2 — it must run.** Something end-to-end has to work. Not slides. Not a mock. Running software.	Live partial demo
22:00	**Code freeze.** Repository locked. Final commit hash recorded.	Repo + README + failure log
22–24	Demos, live code walkthroughs, judging, results.	5-min demo + 2-min pitch

ORGANISER NOTE

Why the Hour 12 curveball matters more than the demo. On a projector every project looks similar. What differs is how fast a new requirement is absorbed — clean boundaries take an hour, hard-coded ones take six. Judges will ask each team what they had to change and how long it took. That answer carries a quarter of the technical score.

06 · SUBMISSIONS

Four Things, No Exceptions

Deliverable	Spec	What it actually reveals
Live demo	5 minutes, running software, real inputs. No pre-recorded video, no slideware.	Whether it exists at all.
“Why shortlist us” pitch	2 minutes, spoken, answering the five questions on the next page.	Whether you understand your own value.
Architecture diagram	One page. Every box, arrow, model call and data store labelled.	Whether you understand your own system.
Failure log	One page. What you tried that failed, what your system still gets wrong, what you'd fix with another week.	Engineering maturity. This is the tie-breaker.

Every member speaks during the walkthrough. Any judge may point at any file and ask the person in front of them to explain it. This single rule removes copied projects more effectively than any plagiarism tool.

07 · THE FIVE QUESTIONS

Your Pitch Must Answer These

These are published in advance deliberately. Preparing for them is the point — a team that can answer all five has thought hard about what they built.

01 **What problem, and who exactly has it?**
Name a real person or role, not “users”. Vague audiences signal a vague project.

02 **What is the non-obvious hard part?**
Every worthwhile project has one thing that turned out to be much harder than expected. Tell us what yours was.

03 **What did you build versus what did the API give you?**
The most important question in the set. Be brutally specific about the boundary.

04 **Why does this break if you remove the AI?**
If it doesn't break, the intelligence is decoration and the score drops accordingly.

05 **What breaks at ten thousand users?**
Cost, latency, rate limits, storage. A thoughtful “here's where it falls over” beats a confident “nothing”.

08 · RUBRIC

How Judges Score

Criterion	Weight	What full marks looks like
Originality	**25%**	Could not have been built in 2023. Judges cannot name an existing product that does this.
Technical depth	**25%**	Substantial work beyond the API call: retrieval, evaluation, guardrails, memory, orchestration.
Working demo	**20%**	Runs live, on real input, without the presenter apologising for anything.
Problem clarity	**15%**	A specific person with a specific pain. The team can say who, in one sentence.
Failure awareness	**15%**	Honest, specific account of what's broken and what it would cost to fix.

IMPORTANT

Visual polish is worth zero points. Announce this at kick-off or half the room will spend six hours on animations and ship nothing. A clean, plain interface loses no marks. An empty one does.

09 · ANTI-PATTERNS

What Gets Rejected Early

Ranked by how often we see it. If your idea appears here, change it before Hour 3.

#	The submission	Why it fails
1	A chatbot over documents with no grounding, no citations and no refusal behaviour	It hallucinates on the first hostile question a judge asks.
2	A thin wrapper: one prompt, one text box, one output	Everyone has the same API key. Nothing here is yours.
3	A resume screener or interview bot	Built at every campus hackathon in the country. Zero originality points.
4	“AI-powered” software where removing the AI changes nothing	Fails the decoration test in question four.
5	A gorgeous interface over a stub that returns hard-coded data	Discovered in the first ninety seconds of the live demo.
6	Something genuinely ambitious that doesn't run at all	Ambition is respected. A dead demo still scores near zero. Scope down at Hour 12.

10 · STARTERS

Directions, Not Answers

Deliberately half-formed. Nobody may submit one of these unchanged — take a direction and turn it into something specific to a person you actually know.

Agents	An agent that runs unattended for the full 24 hours and must survive whatever the organisers break · an agent given a deliberately unreliable set of tools · two agents negotiating on behalf of two humans
Multimodal	A photo of a handwritten timetable becoming a live calendar · a whiteboard photo becoming running code · a code-mixed Hindi-English voice note becoming assigned action items
Retrieval	A knowledge system over documents that contradict each other, which must surface the conflict instead of averaging it · answers that always cite, and refuse when unsure
Evaluation	A system that grades another model's output and flags its own low-confidence cases · a red-team leaderboard where teams try to break each other's guardrails
On-device	Anything useful that runs entirely in the browser or on a low-end phone with the network switched off for the whole demo

“What did you build that the API didn't give you?”

Most teams cannot answer this. The ones who can are the ones we shortlist.
Spend your twenty-four hours making sure you have an answer.

Organising team contact and venue details to be circulated separately. Bring your own laptop, charger and extension board. Sleep is permitted but historically unpopular.